

14.4 Transitive Closure

We have seen that graph traversals can be used to answer basic questions of reachability in a directed graph. In particular, if we are interested in knowing whether there is a path from vertex u to vertex v in a graph, we can perform a DFS or BFS traversal starting at u and observe whether v is discovered. If representing a graph with an adjacency list or adjacency map, we can answer the question of reachability for u and v in $O(n + m)$ time (see Propositions 14.15 and 14.17).

In certain applications, we may wish to answer many reachability queries more efficiently, in which case it may be worthwhile to precompute a more convenient representation of a graph. For example, the first step for a service that computes driving directions from an origin to a destination might be to assess whether the destination is reachable. Similarly, in an electricity network, we may wish to be able to quickly determine whether current flows from one particular vertex to another. Motivated by such applications, we introduce the following definition. The **transitive closure** of a directed graph $\vec{G}$ is itself a directed graph $\vec{G}^*$ such that the vertices of $\vec{G}^*$ are the same as the vertices of $\vec{G}$, and $\vec{G}^*$ has an edge (u, v) , whenever $\vec{G}$ has a directed path from u to v (including the case where (u, v) is an edge of the original $\vec{G}$).

If a graph is represented as an adjacency list or adjacency map, we can compute its transitive closure in $O(n(n + m))$ time by making use of n graph traversals, one from each starting vertex. For example, a DFS starting at vertex u can be used to determine all vertices reachable from u , and thus a collection of edges originating with u in the transitive closure.

In the remainder of this section, we explore an alternative technique for computing the transitive closure of a directed graph that is particularly well suited for when a directed graph is represented by a data structure that supports $O(1)$ -time lookup for the `getEdge( $u, v$ )` method (for example, the adjacency-matrix structure). Let $\vec{G}$ be a directed graph with n vertices and m edges. We compute the transitive closure of $\vec{G}$ in a series of rounds. We initialize $\vec{G}_0 = \vec{G}$. We also arbitrarily number the vertices of $\vec{G}$ as $v_1, v_2, \dots, v_n$. We then begin the computation of the rounds, beginning with round 1. In a generic round k , we construct directed graph $\vec{G}_k$ starting with $\vec{G}_k = \vec{G}_{k-1}$ and adding to $\vec{G}_k$ the directed edge (v_i, v_j) if directed graph $\vec{G}_{k-1}$ contains both the edges (v_i, v_k) and (v_k, v_j) . In this way, we will enforce a simple rule embodied in the proposition that follows.

Proposition 14.18: *For $i = 1, \dots, n$, directed graph $\vec{G}_k$ has an edge (v_i, v_j) if and only if directed graph $\vec{G}$ has a directed path from v_i to v_j , whose intermediate vertices (if any) are in the set $\{v_1, \dots, v_k\}$. In particular, $\vec{G}_n$ is equal to $\vec{G}^*$, the transitive closure of $\vec{G}$.*

Proposition 14.18 suggests a simple algorithm for computing the transitive closure of $\vec{G}$ that is based on the series of rounds to compute each $\vec{G}_k$. This algorithm is known as the **Floyd-Warshall algorithm**, and its pseudocode is given in Code Fragment 14.9. We illustrate an example run of the Floyd-Warshall algorithm in Figure 14.11.

Algorithm FloydWarshall($\vec{G}$):

Input: A directed graph $\vec{G}$ with n vertices

Output: The transitive closure $\vec{G}^*$ of $\vec{G}$

let $v_1, v_2, \dots, v_n$ be an arbitrary numbering of the vertices of $\vec{G}$

$\vec{G}_0 = \vec{G}$

for $k = 1$ to n **do**

$\vec{G}_k = \vec{G}_{k-1}$

for all i, j in $\{1, \dots, n\}$ with $i \neq j$ and $i, j \neq k$ **do**

if both edges (v_i, v_k) and (v_k, v_j) are in $\vec{G}_{k-1}$ **then**

add edge (v_i, v_j) to $\vec{G}_k$ (if it is not already present)

return $\vec{G}_n$

Code Fragment 14.9: Pseudocode for the Floyd-Warshall algorithm. This algorithm computes the transitive closure $\vec{G}^*$ of G by incrementally computing a series of directed graphs $\vec{G}_0, \vec{G}_1, \dots, \vec{G}_n$, for $k = 1, \dots, n$.

From this pseudocode, we can easily analyze the running time of the Floyd-Warshall algorithm assuming that the data structure representing G supports methods `getEdge` and `insertEdge` in $O(1)$ time. The main loop is executed n times and the inner loop considers each of $O(n^2)$ pairs of vertices, performing a constant-time computation for each one. Thus, the total running time of the Floyd-Warshall algorithm is $O(n^3)$. From the description and analysis above we may immediately derive the following proposition.

Proposition 14.19: Let $\vec{G}$ be a directed graph with n vertices, and let $\vec{G}$ be represented by a data structure that supports lookup and update of adjacency information in $O(1)$ time. Then the Floyd-Warshall algorithm computes the transitive closure $\vec{G}^*$ of $\vec{G}$ in $O(n^3)$ time.

Performance of the Floyd-Warshall Algorithm

Asymptotically, the $O(n^3)$ running time of the Floyd-Warshall algorithm is no better than that achieved by repeatedly running DFS, once from each vertex, to compute the reachability. However, the Floyd-Warshall algorithm matches the asymptotic bounds of the repeated DFS when a graph is dense, or when a graph is sparse but represented as an adjacency matrix. (See Exercise R-14.13.)

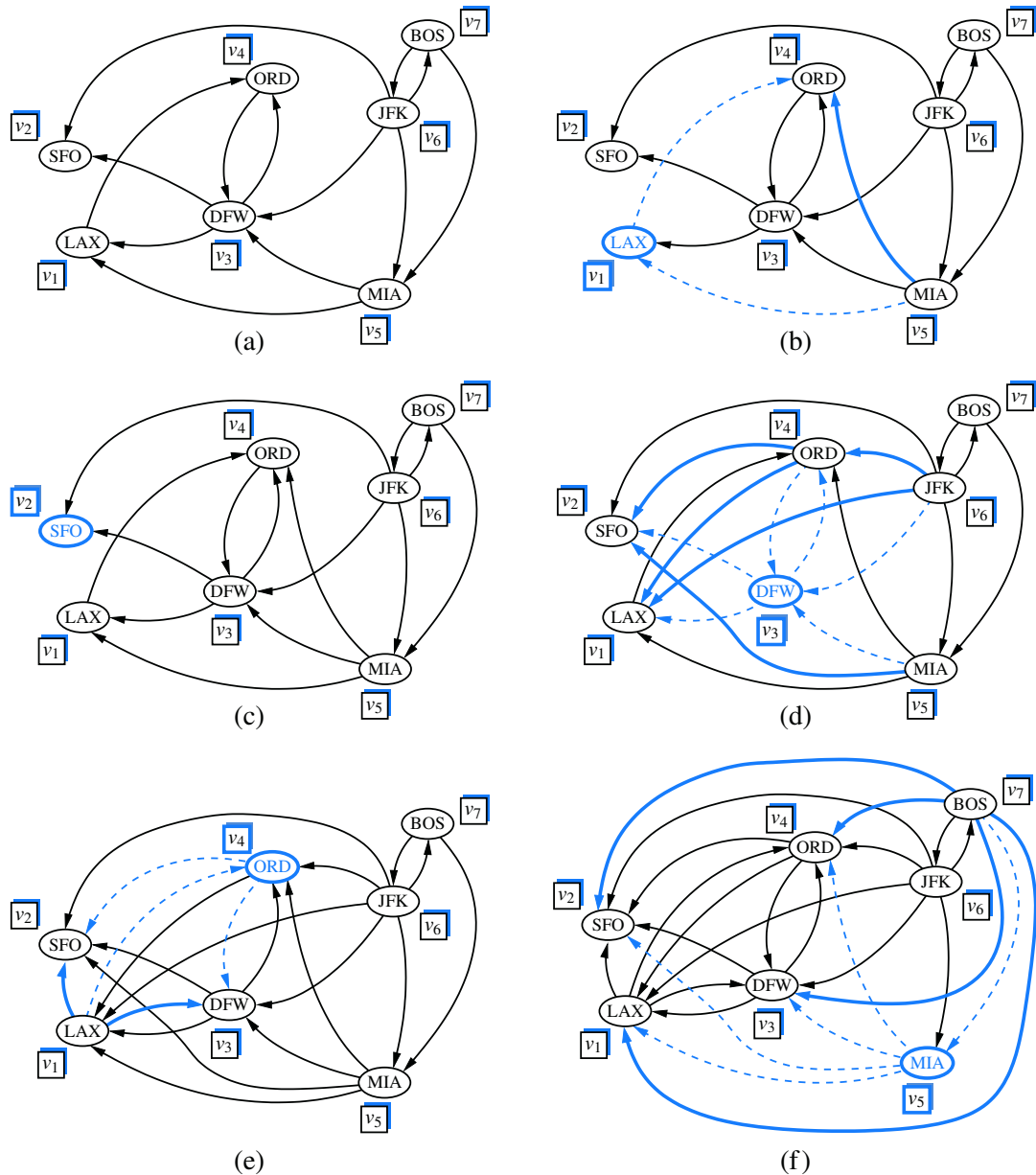


Figure 14.11: Sequence of directed graphs computed by the Floyd-Warshall algorithm: (a) initial directed graph $\vec{G} = \vec{G}_0$ and numbering of the vertices; (b) directed graph $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Note that $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. If directed graph $\vec{G}_{k-1}$ has the edges (v_i, v_k) and (v_k, v_j) , but not the edge (v_i, v_j) , in the drawing of directed graph $\vec{G}_k$, we show edges (v_i, v_k) and (v_k, v_j) with dashed lines, and edge (v_i, v_j) with a thick line. For example, in (b) existing edges (MIA, LAX) and (LAX, ORD) result in new edge (MIA, ORD) .

The importance of the Floyd-Warshall algorithm is that it is much easier to implement than repeated DFS, and much faster in practice because there are relatively few low-level operations hidden within the asymptotic notation. The algorithm is particularly well suited for the use of an adjacency matrix, as a single bit can be used to designate the reachability modeled as an edge (u, v) in the transitive closure.

However, note that repeated calls to DFS results in better asymptotic performance when the graph is sparse and represented using an adjacency list or adjacency map. In that case, a single DFS runs in $O(n + m)$ time, and so the transitive closure can be computed in $O(n^2 + nm)$ time, which is preferable to $O(n^3)$.

Java Implementation

We will conclude with a Java implementation of the Floyd-Warshall algorithm, as presented in Code Fragment 14.10. Although the pseudocode for the algorithm describes a series of directed graphs $\vec{G}_0, \vec{G}_1, \dots, \vec{G}_n$, we directly modify the original graph, repeatedly adding new edges to the closure as we progress through rounds of the Floyd-Warshall algorithm.

Also, the pseudocode for the algorithm describes the loops based on vertices being indexed from 0 to $n - 1$. With our graph ADT, we prefer to use Java's for-each loop syntax directly on the vertices of the graph. Therefore, in Code Fragment 14.10, variables i , j , and k are references to vertices, not integer indices into the sequence of vertices.

Finally, we make one additional optimization in the Java implementation, relative to the pseudocode, by not bothering to iterate through values of j unless we have verified that edge (i, k) exists in the current version of the closure.

```

1  /** Converts graph g into its transitive closure. */
2  public static <V,E> void transitiveClosure(Graph<V,E> g) {
3      for (Vertex<V> k : g.vertices())
4          for (Vertex<V> i : g.vertices())
5              // verify that edge (i,k) exists in the partial closure
6              if (i != k && g.getEdge(i,k) != null)
7                  for (Vertex<V> j : g.vertices())
8                      // verify that edge (k,j) exists in the partial closure
9                      if (i != j && j != k && g.getEdge(k,j) != null)
10                         // if (i,j) not yet included, add it to the closure
11                         if (g.getEdge(i,j) == null)
12                             g.insertEdge(i, j, null);
13  }
```

Code Fragment 14.10: Java implementation of the Floyd-Warshall algorithm.